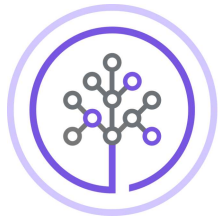


Hands-on Lab: Create a DAG for Apache Airflow with PythonOperator



Skills
Network

Estimated time needed: **40** minutes

Introduction

In this lab, you will explore the Apache Airflow web user interface (UI). You will then create a Direct Acyclic Graph (DAG) using PythonOperator and finally run it through the Airflow web UI.

Objectives

After completing this lab, you will be able to:

- Explore the Airflow Web UI
- Create a DAG with PythonOperator
- Submit a DAG and run it through the Web UI

Prerequisite

Please ensure that you have completed the reading on the [Airflow DAG Operators](#) before proceeding with this lab. You should be familiar with Python input and output (I/O) operations and request packages to complete this lab.

About Skills Network Cloud IDE

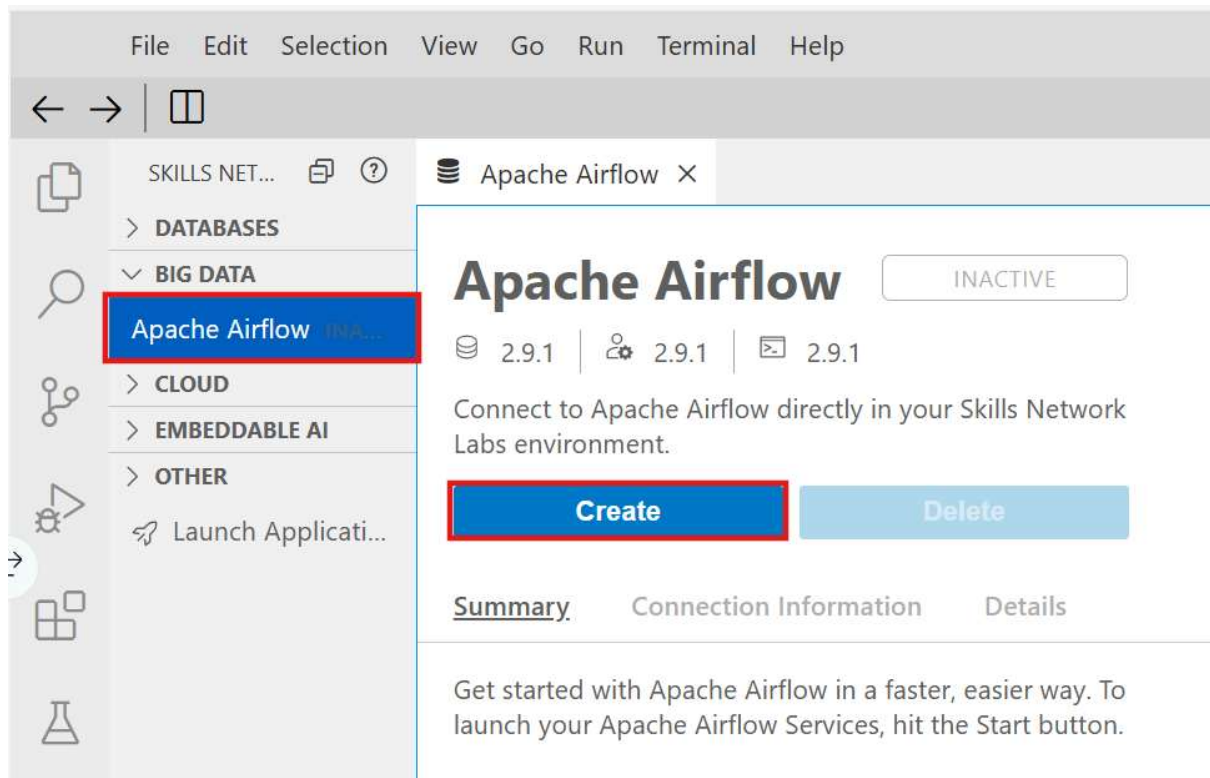
Skills Network Cloud IDE (based on Theia and Docker) provides an environment for hands-on labs for course and project-related labs. Theia is an open-source IDE (Integrated Development Environment) that can be run on a desktop or on the cloud. To complete this lab, you will be using the Cloud IDE based on Theia, running in a Docker container.

Important notice about this lab environment

Please be aware that sessions for this lab environment are not persistent. A new environment is created for you every time you connect to this lab. Any data you may have saved in an earlier session will get lost. To avoid losing your data, please plan to complete these labs in a single session.

Exercise 1: Start Apache Airflow

1. Click on **Skills Network Toolbox**.
2. From the **BIG DATA** section, click **Apache Airflow**.
3. Click **Create** to start the Apache Airflow.



Note: Please be patient, it will take a few minutes for Airflow to start. If there is an error starting Airflow, please restart it.

Exercise 2: Open the Airflow Web UI

1. When Airflow starts successfully, you should see an output similar to the one below. Once **Apache Airflow** has started, click on the highlighted icon to open **Apache Airflow Web UI** in the new window.

Apache Airflow

ACTIVE

2.9.1 | 2.9.1 | 2.9.1

Connect to Apache Airflow directly in your Skills Network Labs environment.

[Start](#) [Stop](#)

[Summary](#) [Connection Information](#) [Details](#)

Your Apache Airflow Services are now ready to use and available with the following login credentials. For more details on how to navigate Apache Airflow, please check out the Details section.

Username:

Password:

You can manage Apache Airflow via:

[Airflow Webserver](#) 

You should land on a page that looks like this.

Airflow									
DAGs Cluster Activity Datasets Security Browse Admin Docs									
03:53 UTC - Log In									
☐	example_bash_operator	airflow	00:00:00	1	2024-05-29, 00:00:00	1			
☐	example_branch_datetime_operator	airflow	@daily	1	2024-05-29, 00:00:00	1			
☐	example_branch_datetime_operator_2	airflow	@daily	1	2024-05-29, 00:00:00	1			
☐	example_branch_datetime_operator_3	airflow	@daily	1	2024-05-29, 00:00:00	1			
☐	example_branch_dop_operator_v3	airflow	*1 * * * *	1	2024-05-30, 03:51:00	1			
☐	example_branch_labels	airflow	@daily	1	2024-05-29, 00:00:00	1			
☐	example_branch_operator	airflow	@daily	1	2024-05-29, 00:00:00	1			
☐	example_branch_python_operator_decorator	airflow	@daily	1	2024-05-29, 00:00:00	1			

Exercise 3: Create a DAG with PythonOperator

Next, you will create a DAG, which will define a pipeline of tasks, such as `extract`, `transform`, `load`, and `check` with PythonOperator.

1. Create a `DAG` file, `my_first_dag.py`, which will run daily. To Create a new file choose `File` -> `New File` and name it as `my_first_dag.py`. The `my_first_dag.py` file defines tasks `execute_extract`, `execute_transform`, `execute_load`, and `execute_check` to call the respective Python functions.



python



```
# Import the libraries
from datetime import timedelta
# The DAG object; we'll need this to instantiate a DAG
from airflow.models import DAG
# Operators; you need this to write tasks!
from airflow.operators.python import PythonOperator

# This makes scheduling easy
from airflow.utils.dates import days_ago

# Define the path for the input and output files
input_file = '/etc/passwd'
extracted_file = 'extracted-data.txt'
transformed_file = 'transformed.txt'
output_file = 'data_for_analytics.csv'

def extract():
    global input_file
    print("Inside Extract")
    # Read the contents of the file into a string
    with open(input_file, 'r') as infile, \
        open(extracted_file, 'w') as outfile:
        for line in infile:
            fields = line.split(':')
            if len(fields) >= 6:
                field_1 = fields[0]
                field_3 = fields[2]
                field_6 = fields[5]
                outfile.write(field_1 + ":" + field_3 + ":" + field_6 + "\n")

def transform():
    global extracted_file, transformed_file
    print("Inside Transform")
    with open(extracted_file, 'r') as infile, \
        open(transformed_file, 'w') as outfile:
        for line in infile:
            processed_line = line.replace(':', ',')
            outfile.write(processed_line + '\n')

def load():
    global transformed_file, output_file
    print("Inside Load")
    # Save the array to a CSV file
    with open(transformed_file, 'r') as infile, \
        open(output_file, 'w') as outfile:
```

```

def check():
    global output_file
    print("Inside Check")
    # Save the array to a CSV file
    with open(output_file, 'r') as infile:
        for line in infile:
            print(line)

# You can override them on a per-task basis during operator initialization
default_args = {
    'owner': 'Your name',
    'start_date': days_ago(0),
    'email': ['your email'],
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

# Define the DAG
dag = DAG(
    'my-first-python-etl-dag',
    default_args=default_args,
    description='My first DAG',
    schedule_interval=timedelta(days=1),
)

# Define the task named execute_extract to call the `extract` function
execute_extract = PythonOperator(
    task_id='extract',
    python_callable=extract,
    dag=dag,
)

# Define the task named execute_transform to call the `transform` function
execute_transform = PythonOperator(
    task_id='transform',
    python_callable=transform,
    dag=dag,
)

# Define the task named execute_load to call the `load` function
execute_load = PythonOperator(
    task_id='load',
    python_callable=load,
    dag=dag,
)

# Define the task named execute_load to call the `load` function
execute_check = PythonOperator(

```

```
        dag=dag,  
    )  
  
    # Task pipeline  
    execute_extract \> execute_transform \> execute_load \> execute_check
```

Exercise 4: Submit a DAG

Submitting a DAG is as simple as copying the DAG Python file into the `dags` folder in the `AIRFLOW_HOME` directory.

1. Open a terminal and run the command below to set the `AIRFLOW_HOME`.

 php 

```
export AIRFLOW_HOME=/home/project/airflow
echo $AIRFLOW_HOME
```

```
theia@theiadocker-lavanyas: /home/project ×
theia@theiadocker-lavanyas:/home/project$ echo $AIRFLOW_HOME
/home/project/airflow
```

2. Run the command below to submit the DAG that was created in the previous exercise.

 plaintext 

```
cp my_first_dag.py $AIRFLOW_HOME/dags
```

3. Verify that your DAG actually got submitted.
4. Run the command below to list out all the existing DAGs.

 plaintext 

```
airflow dags list
```

5. Verify that `my-first-python-etl-dag` is a part of the output.

 plaintext 

```
airflow dags list|grep "my-first-python-etl-dag"
```

6. You should see your DAG name in the output.

7. Run the command below to list out all the tasks in `my-first-python-etl-dag`.



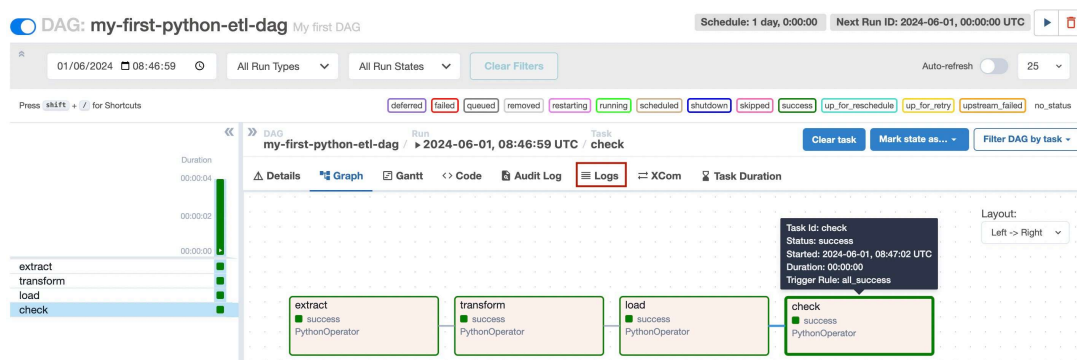
plaintext



```
airflow tasks list my-first-python-etl-dag
```

8. You should see all the four tasks in the output.

9. You can run the task from the Web UI. You can check the logs of the tasks by clicking the individual task in the Graph view.



Practice exercise

Write a DAG named `ETL_Server_Access_Log_Processing` that will extract a file from a remote server and then transform the content and load it into a file.

The file URL is given below: <https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DB0250EN-SkillsNetwork/labs/Apache%20Airflow/Build%20a%20DAG%20using%20Airflow/web-server-access-log.txt>

The server access log file contains these fields.

a. `timestamp` - TIMESTAMP b. `latitude` - float c. `longitude` - float d. `visitorid` - char(37) e. `accessed_from_mobile` - boolean f. `browser_code` - int

Tasks

1. Add tasks in the DAG file to download the file, read the file, and extract the fields `timestamp` and `visitorid` from the `web-server-access-log.txt`.
2. Capitalize the `visitorid` for all the records and store it in a local variable.
3. Load the data into a new file `capitalized.txt`.
4. Create the imports block.
5. Create the DAG Arguments block. You can use the default settings.
6. Create the DAG definition block. The DAG should run daily.
7. Create the tasks extract, transform, and load to call the Python script.
8. Create the task pipeline block.
9. Submit the DAG.
10. Verify if the DAG is submitted.

▶ [Click here for hint](#)

► [Click here for the solution.](#)

Authors

[Lavanya T S](#)

Other Contributors

Rav Ahuja

© IBM Corporation. All rights reserved.

Practice exercise

Write a DAG named `ETL_Server_Access_Log_Processing` that will extract a file from a remote server and then transform the content and load it into a file.

The file URL is given below: <https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DB0250EN-SkillsNetwork/labs/Apache%20Airflow/Build%20a%20DAG%20using%20Airflow/web-server-access-log.txt>

The server access log file contains these fields.

a. `timestamp` - TIMESTAMP b. `latitude` - float c. `longitude` - float d. `visitorid` - char(37) e. `accessed_from_mobile` - boolean f. `browser_code` - int

Tasks

1. Add tasks in the DAG file to download the file, read the file, and extract the fields `timestamp` and `visitorid` from the `web-server-access-log.txt`.
2. Capitalize the `visitorid` for all the records and store it in a local variable.

3. Load the data into a new file `capitalized.txt`.
4. Create the imports block.
5. Create the DAG Arguments block. You can use the default settings.
6. Create the DAG definition block. The DAG should run daily.
7. Create the tasks extract, transform, and load to call the Python script.
8. Create the task pipeline block.
9. Submit the DAG.
10. Verify if the DAG is submitted.

▼ **Click here for hint.**

Follow the example code given in the lab and make necessary changes to create the new DAG.

▼ [Click here for the solution.](#)

Create a new file by going to **File -> New File** from the menu and name it as `ETL_Server_Access_Log_Processing.py`. Copy the code below in the python file. This will contain your DAG with five tasks:

- download
- execute_extract
- execute_transform
- execute_load
- execute_check



python



```
# Import the libraries
from datetime import timedelta
# The DAG object; we'll need this to instantiate a DAG
from airflow.models import DAG
# Operators; you need this to write tasks!
from airflow.operators.python import PythonOperator
from airflow.operators.bash_operator import BashOperator

# This makes scheduling easy
from airflow.utils.dates import days_ago
import requests

# Define the path for the input and output files
input_file = 'web-server-access-log.txt'
extracted_file = 'extracted-data.txt'
transformed_file = 'transformed.txt'
output_file = 'capitalized.txt'

def download_file():
    url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.com"
    # Send a GET request to the URL
    with requests.get(url, stream=True) as response:
        # Raise an exception for HTTP errors
        response.raise_for_status()
        # Open a local file in binary write mode
        with open(input_file, 'wb') as file:
            # Write the content to the local file in chunks
            for chunk in response.iter_content(chunk_size=8192):
                file.write(chunk)
    print(f"File downloaded successfully: {input_file}")

def extract():
    global input_file
    print("Inside Extract")
    # Read the contents of the file into a string
    with open(input_file, 'r') as infile, \
        open(extracted_file, 'w') as outfile:
        for line in infile:
            fields = line.split('#')
            if len(fields) >= 4:
                field_1 = fields[0]
                field_4 = fields[3]
                outfile.write(field_1 + "#" + field_4 + "\n")

def transform():
```

```

with open(extracted_file, 'r') as infile, \
    open(transformed_file, 'w') as outfile:
    for line in infile:
        processed_line = line.upper()
        outfile.write(processed_line + '\n')

def load():
    global transformed_file, output_file
    print("Inside Load")
    # Save the array to a CSV file
    with open(transformed_file, 'r') as infile, \
        open(output_file, 'w') as outfile:
        for line in infile:
            outfile.write(line + '\n')

def check():
    global output_file
    print("Inside Check")
    # Save the array to a CSV file
    with open(output_file, 'r') as infile:
        for line in infile:
            print(line)

# You can override them on a per-task basis during operator initialization
default_args = {
    'owner': 'Your name',
    'start_date': days_ago(0),
    'email': ['your email'],
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

# Define the DAG
dag = DAG(
    'my-first-python-etl-dag',
    default_args=default_args,
    description='My first DAG',
    schedule_interval=timedelta(days=1),
)

# Define the task named download to call the `download_file` function
download = PythonOperator(
    task_id='download',
    python_callable=download_file,
    dag=dag,
)

# Define the task named execute_extract to call the `extract` function

```

```

python_callable=extract,
dag=dag,
)

# Define the task named execute_transform to call the `transform` func
execute_transform = PythonOperator(
    task_id='transform',
    python_callable=transform,
    dag=dag,
)

# Define the task named execute_load to call the `load` function
execute_load = PythonOperator(
    task_id='load',
    python_callable=load,
    dag=dag,
)

# Define the task named execute_load to call the `load` function
execute_check = PythonOperator(
    task_id='check',
    python_callable=check,
    dag=dag,
)

# Task pipeline
download >> execute_extract >> execute_transform >> execute_load >> ex

```

Copy the DAG file into the **dags** directory.


bash


```
cp ETL_Server_Access_Log_Processing.py $AIRFLOW_HOME/dags
```

 Run

Verify if the DAG is submitted by running the following command.


bash


```
airflow dags list | grep etl-server-logs-dag
```

 Run

If the DAG didn't get imported properly, you can check the error using the following command.



bash



```
airflow dags list-import-errors
```

▶ Run

Authors

[Lavanya T S](#)

Other Contributors

Rav Ahuja

© IBM Corporation. All rights reserved.

← Exercise 4: Submit a DAG